

mspix File Format Specification

Version: 1.0.5

1. Scope

The mspix file format is an open storage and data-access format for mass spectrometry imaging (MSI) data. It is intended for large-pixel-count images, such as those produced using time-of-flight instruments, including fast mass microscopy and time-of-flight secondary ion mass spectrometry (TOF-SIMS). The format is optimized for large-pixel-count datasets and is recommended for images exceeding five million pixels; it is capable of efficiently representing images of more than one billion pixels. The format is also well-suited for targeted MSI experiments in which only a subset of spectral channels are recorded per pixel.

mspix is designed around three properties common to high-pixel-count TOF-based MSI data:

1. Data sparsity: Most spectral channels in a given pixel have zero intensity.
2. A single shared mass axis: All pixels share one one-dimensional spectral channel list.
3. A rectangular pixel grid.

mspix is not a general-purpose MSI format and is not intended to replace existing formats such as imzML for data types that do not satisfy the above properties. Data types outside its scope include ion-mobility data, multi-tandem-MS data with non-shared secondary axes, non-rectangular or non-gridded pixel arrangements, and continuous (profile) spectral data.

2. Normative References

- **IEEE 754-2019.** IEEE Standard for Floating-Point Arithmetic.
- **RFC 8259.** The JavaScript Object Notation (JSON) Data Interchange Format. Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc8259>
- **HDF5 File Format Specification.** The HDF Group. https://docs.hdfgroup.org/hdf5/develop/_f_m_t3.html
- **Semantic Versioning 2.0.0.** Preston-Werner, T. <https://semver.org>

3. Format Overview

An mspix image exists in one of two equivalent representations:

1. Loose representation: a directory named with a `.mspix` extension, containing a set of specifically named constituent files.
2. Packed representation: a single HDF5 file named with a `.mspix` extension.

Both representations encode the same information. Implementations should detect whether a given `.mspix` path is a directory (loose representation) or a regular file (packed representation) to determine how to load it.

The packed representation is recommended for most use cases, as it provides native compression, chunking, and broad library support across programming languages. It is also a single file, rather than a folder of files. Unlike `imzML`, the loose `mspix` representation provides no UUID or checksum to validate that the separate files have not been altered or correctly correspond to one another. The loose representation should only be preferred when:

1. an HDF5 library is unavailable in the target programming environment;
2. the implementation requires memory-mapping or custom parallel access patterns beyond those supported by available HDF5 libraries; or
3. simplicity of implementation is a priority, such as during instrument development or early-stage tooling.

4. Loose Representation

4.1 Folder Structure

The loose `mspix` dataset directory with the `.mspix` extension must contain all of the following files:

Filename	Description
<code>metadata.json</code>	Image metadata
<code>pixel_intensities.xxx</code>	Per-pixel total intensity
<code>pixel_channels.uXX</code>	Per-pixel nonzero channel count
<code>intensities.xxx</code>	Packed nonzero channel intensities
<code>indices.uXX</code>	Packed nonzero channel indices

where `.xxx` and `.uXX` are numeric type suffixes defined in Appendix A. The directory may contain additional files alongside these required files.

4.2 `metadata.json`

The `metadata.json` file is a UTF-8-encoded, JSON-compliant text file containing metadata for the image. The following keys are required:

Key	JSON type	Description
<code>mspix_version</code>	String	Semantic version of the <code>mspix</code> specification used to produce this file, conforming to Semantic Versioning 2.0.0. For files produced under this version of the specification, the value must be <code>"1.0.5"</code> .
<code>image_width_pixels</code>	Number	Width of the image in pixels. A positive integer representable as a 64-bit IEEE 754 floating-point

Key	JSON type	Description
<code>image_height_pixels</code>	Number	value. Height of the image in pixels. A positive integer representable as a 64-bit IEEE 754 floating-point value.
<code>spectral_channels</code>	Array of numbers	The list of spectral channel values constituting the mass axis. The list should be ordered from small to large; although unordered lists should be accepted by implementations. Values are most commonly m/z values but may be raw times-of-flight or other representations. All values must be representable as 64-bit IEEE 754 floating-point numbers. The maximum valid index value in <code>indices.uXX</code> should be equal to one less than the length of this array (zero-indexing). When values are not m/z values, a corresponding m/z list should be provided as an additional metadata entry.
<code>spectral_intensities</code>	Array of numbers	The total accumulated intensity across all pixels for each spectral channel. This array must have the same length as <code>spectral_channels</code>. All values must be representable as 64-bit IEEE 754 floating-point numbers. This data is redundant and may be derived from <code>indices.uXX</code> and <code>intensities.xxx</code>, but must be present to support efficient spectrum inspection without full data traversal.

The following additional specific metadata keys are recommended but not required:

Key(s)	JSON type	Description
<code>pixel_width_um</code> and <code>pixel_height_um</code>	Number	The spatial size of each pixel in micrometers (abbreviated <code>um</code> intentionally to enable ease of search and typing instead of μm).
<code>image_width_mm</code> and <code>image_height_mm</code>	Number	Physical image dimensions in millimeters.

The `metadata.json` file is extensible; additional keys beyond those specified above should be included as needed; however, implementations should accept files without such

additional keys. An example of a `metadata.json` file is provided in Appendix B.

4.3 `pixel_intensities.xxx`

The `pixel_intensities.xxx` is a flat, little-endian binary file. The numeric type is specified by the `.xxx` extension (Appendix A); valid types are `.u8`, `.u16`, `.u32`, `.u64`, `.f32`, and `.f64`.

The file must contain exactly `image_width_pixels × image_height_pixels` values stored in row-major order. The value at position `pixel_index` represents the total ion intensity for that pixel and should equal the sum of all nonzero channel intensities for that pixel as derivable from `intensities.xxx` and `indices.uXX`.

This file is provided to enable efficient total-ion image display, region-of-interest selection, and intensity normalization without requiring traversal of the full spectral data.

4.4 `pixel_channels.uXX`

The `pixel_channels.uXX` is a flat, little-endian binary file. The numeric type is specified by the `.uXX` extension (Appendix A); only unsigned integer types are valid: `.u8`, `.u16`, `.u32`, `.u64`.

The file must contain exactly `image_width_pixels × image_height_pixels` values stored in row-major order. Each value must be an unsigned integer equal to the count of entries stored for the corresponding pixel in `indices.uXX` and `intensities.xxx`.

This file encodes two distinct zero states that must be distinguished:

- A value of zero indicates an unsampled pixel, where no spectrum was acquired at the pixel's location.
- A value of one, where the corresponding entry in `intensities.xxx` has a value of zero, indicates a sampled pixel with no detected ions. The index value of this entry may be any valid index into `spectral_channels`.

This distinction exists because `mspix` images are constrained to rectangular grids, making it necessary to pad regions that were not physically sampled. The offset table is unaffected by this convention.

The offset table for accessing pixel data in `indices.uXX` and `intensities.xxx` is derived by cumulative summation of the `pixel_channels.uXX` file.

4.5 `intensities.xxx`

The `intensities.xxx` is a flat, little-endian binary file. The numeric type is specified by the `.xxx` extension (Appendix A); valid types are `.u8`, `.u16`, `.u32`, `.u64`, `.f32`, and `.f64`.

The file contains the intensities of all nonzero spectral channels across all pixels, concatenated in pixel index order. Pixel boundaries within this file are determined by the offset table derived from `pixel_channels.uXX`. Each value corresponds to the intensity of the spectral channel identified by the value at the same position in `indices.uXX`.

The total number of values in `intensities.xxx` must equal the total number of values in `indices.uXX`, and must equal the sum of all values in `pixel_channels.uXX`.

Zero-intensity entries must be omitted, with one exception: a sampled pixel with no detected ions must be represented by exactly one entry with an intensity value of zero. No other zero-intensity entries are permitted.

4.6 indices.uXX

The `indices.uXX` is a flat, little-endian binary file. The numeric type is specified by the `.uXX` extension (Appendix A); only unsigned integer types are valid: `.u8`, `.u16`, `.u32`, `.u64`. The integer width should be chosen to be the smallest type sufficient to represent the largest index value; the maximum valid index must be strictly less than the length of `spectral_channels`.

Each value is the zero-based index of a nonzero spectral channel for a pixel, indexing into the `spectral_channels` array in `metadata.json`. The intensity of each indexed channel is found at the same position in `intensities.xxx`.

The total number of values in `indices.uXX` must equal the total number of values in `intensities.xxx`.

Indices within a single pixel's entry should be stored in ascending order. Ascending order enables efficient channel lookup within a pixel and is the convention used by the reference implementation.

5. Packed Representation

The packed representation is a single HDF5 file with a `.mspix` extension. The file must contain the HDF5 datasets named and described below. Additional HDF5 datasets or groups may be present.

Dataset name	Dimensionality	Description
meta	Scalar	UTF-8 JSON metadata string
pixel_intensities	2D (H, W)	Per-pixel total intensity
pixel_channels	2D (H, W)	Per-pixel entry count
intensities	1D	Packed channel intensities
indices	1D	Packed channel indices

5.1 meta

A scalar HDF5 dataset containing a single UTF-8-encoded string. The string content must conform to all requirements of the `metadata.json` file described in Section 4.2.

5.2 pixel_intensities

A two-dimensional HDF5 dataset with dimensions (`image_height_pixels`, `image_width_pixels`), where the first dimension corresponds to rows and the second to columns, consistent with row-major layout. The datatype must be one of the compatible types listed in Appendix A.

This dataset is the two-dimensional equivalent of the flat `pixel_intensities.xxx` file described in Section 4.3. All value requirements specified in Section 4.3 apply.

5.3 pixel_channels

A two-dimensional HDF5 dataset with dimensions (`image_height_pixels`, `image_width_pixels`). The datatype must be an unsigned integer type (Appendix A).

This dataset is the two-dimensional equivalent of the flat `pixel_channels.uXX` file described in Section 4.4. All value requirements specified in Section 4.4 apply.

5.4 intensities

A one-dimensional HDF5 dataset. The datatype must be one of the compatible types listed in Appendix A. All content and constraints are identical to those described for `intensities.xXX` in Section 4.5.

5.5 indices

A one-dimensional HDF5 dataset. The datatype must be an unsigned integer type (Appendix A). All content and constraints are identical to those described for `indices.uXX` in Section 4.6.

5.6 Compression

If compression is desired, HDF5's built-in per-dataset compression options (e.g., DEFLATE/gzip, LZ4) are recommended. The `indices` and `intensities` datasets are typically the largest and benefit most from compression.

Compression of the HDF5 file as a whole (e.g., by gzip-compressing the `.mspix` file) is not recommended, as it prevents random access to the data.

6. Image Orientation and Pixel Ordering

All `mspix` images use a row-major pixel layout. Pixel (row 0, column 0) is located at the top-left corner of the image. Row index increases downward; column index increases to the right. The linear pixel index is:

$$\text{pixel_index} = \text{row} \times \text{image_width_pixels} + \text{column}$$

This ordering applies to the flat binary files of the loose representation and, equivalently, to the two-dimensional HDF5 datasets of the packed representation (first dimension = row, second dimension = column).

The pixel layout is defined independently of scan trajectory during acquisition (e.g., serpentine, typewriter, or unidirectional raster). Scan trajectory information, if needed, should be stored in `metadata.json`.

6.1 Offset Table and Data Access

The entries for a pixel in `indices` and `intensities` are located using an offset table derived from `pixel_channels`. For the pixel at `pixel_index i`, the offset into `indices` and `intensities` is:

$\text{offset}(i) = \text{pixel_channels}[0] + \text{pixel_channels}[1] + \dots + \text{pixel_channels}[i - 1]$

with $\text{offset}(0) = 0$. The pixel's entries occupy positions $\text{offset}(i)$ through $\text{offset}(i) + \text{pixel_channels}[i] - 1$ (inclusive) in both arrays. A pixel with $\text{pixel_channels}[i] = 0$ has no entries and requires no access.

Figure 1 illustrates this structure for a minimal example.

Figure 1. Data layout for a 2x2 image with 6 spectral channels.

`pixel_channels.uXX` (4 values, row-major)

2	3	0	1
---	---	---	---

px 0 px 1 px 2 px 3
 (r0,c0) (r0,c1) (r1,c0) (r1,c1)

Offset table (cumulative sum of `pixel_channels.uXX` file, length = $N_{\text{pixels}} + 1 = 5$):

0	2	5	5	6
---	---	---	---	---

px 2 has a channels value of 0, meaning it has no mass information (was unsampled)

`indices.uXX` (6 values total)

1	4	0	2	5	3
---	---	---	---	---	---

px 0 px 1 px 3
 (2 vals) (3 vals) (1 val)

px 2 has 0 values and so indices goes directly from px 1 to px 3.

`intensities.xXX` (6 values total)

70	90	50	30	80	60
----	----	----	----	----	----

px 0 px 1 px 3
 (2 vals) (3 vals) (1 val)

spectral_channels in metadata.json:

index: [0 , 1 , 2 , 3 , 4 , 5]
 value: [100.0, 200.0, 300.0, 400.0, 500.0, 600.0] (m/z)

Reading pixel 0 (row 0, col 0):

offset = 0, count = 2
 channels: spectral_channels[1]=200.0, spectral_channels[4]=500.0
 intensities: 70, 90

7. Applicability and Limitations

mspix is designed around constraints common to high-pixel-count TOF-based MSI experiments. Data not satisfying these constraints cannot be represented in mspix without data loss.

1. Sparse processed data only: mspix assumes data have been processed to remove noise and zero-intensity channels. Continuous (profile) and event-based TOF data must be converted to processed form prior to encoding in mspix.
2. Rectangular images only: mspix images must be rectangular. Non-rectangular imaged areas may be represented by padding unsampled pixels with a `pixel_channels` value of zero. Such empty pixels consume minimal storage.
3. Single one-dimensional spectral axis: A single shared `spectral_channels` axis must apply to all pixels. Data requiring multiple or non-shared axes — including some ion-mobility and multi-tandem-MS datasets — cannot be efficiently represented in mspix 1.0.5.

Appendix A: Valid Numeric Type Extensions

In the loose representation, each binary file carries a type-indicating extension of the form `.xxx`, where `x` is the numeric kind and `xx` is the bit width. In the packed representation, the datatype is encoded in the HDF5 dataset metadata and no separate suffix is needed.

Extension	Kind	Bit width	Valid for
<code>.u8</code>	Unsigned integer	8	All binary files
<code>.u16</code>	Unsigned integer	16	All binary files
<code>.u32</code>	Unsigned integer	32	All binary files
<code>.u64</code>	Unsigned integer	64	All binary files
<code>.f32</code>	IEEE 754 floating point	32	<code>pixel_intensities</code> , <code>intensities</code> only
<code>.f64</code>	IEEE 754 floating point	64	<code>pixel_intensities</code> , <code>intensities</code> only

All binary files in the loose representation must use little-endian byte order.

Appendix B: Example metadata.json

The following is an abbreviated example of a conforming `metadata.json`. An ellipsis in the array values indicates omitted entries and would not appear in a valid file.

```
{  
  "mspix_version": "1.0.5",  
  ...  
}
```



```
"image_width_pixels": 2412,  
"image_height_pixels": 2411,  
"spectral_channels": [1.000, 1.005, 1.010, ..., 999.990, 999.995,  
    1000.000],  
"spectral_intensities": [217, 151, 307, ..., 58, 30, 176],  
"original_file": "C:\\Users\\user\\Desktop\\example.tpx3",  
"image_width_mm": 2.0025000934838317,  
"image_height_mm": 2.001499891281128,  
"acquisition_time_ns": 173700783473,  
"tof_pulse_count": 3358385  
}
```